2026年下半年系统架构设计师考前几页纸

1、信息系统的分类

信息系统的分类	关键点
业务处理系统【TPS】	早期最初级的信息系统【20世纪50-60年代】。功能：数据输入、数据处理【批处理、OLTP】、数据库维护、文件报表产生。
管理信息系统【MIS】	高度集成化的人机信息系统。 金字塔结构：分多个层级。
决策支持系统【DSS】	由语言系统、知识系统和问题处理系统组成。 用于辅助决策、支持决策。
专家系统【ES】	知识表示+知识推理技术=专家系统。人工智能的一个重要分支。
办公自动化系统【OAS】	由计算机设备、办公设备、数据通信及网络设备、软件系统组成。
企业资源计划【ERP】	打通供应链，集成，整合。

2、专家系统【ES】：

【专家系统（Expert System，ES）】是一个智能计算机程序系统，其内部含有某个领域具有专家水平的大量知识与经验，能够利用人类专家的知识和解决问题的方法来处理该领域的问题。

知识库：存储求解实际问题的领域知识。

综合数据库：存储问题的状态描述、中间结果、求解过程的记录等信息。

推理机：实质是【规则解释器】。

知识获取：两方面功能——知识的编辑求精及知识自学习。

解释程序：面向用户服务的。

3、影响软件质量的3组因素：

（1）产品修改

可理解性（我能理解它吗？）

可维修性（我能修复它吗？）

灵活性（我能改变它吗？）

可测试性（我能测试它吗？）

（2）产品转移

可移植性（我能在另一台机器上使用它吗？）

可再用性（我能再用它的某些部分吗？）

互运行性（我能把它和另一个系统结合吗？）

（3）产品运行

正确性（它按我的需要工作吗？）

健壮性（对意外环境它能适当地响应吗？）

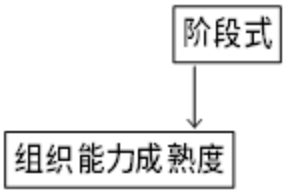
效率（完成预定功能时它需要的计算机资源多吗？）

完整性（它是安全的吗？）

可用性（我能使用它吗？）

风险（能按预定计划完成它吗？）

4、软件能力成熟度模型集成（CMMI）



成熟度等级	特点
初始级【L1】	随意且混乱、组织成功依赖于个人能力
已管理级【L2】	项目级可重复【建立了项目级的控制过程】
已定义级【L3】	组织级，文档化标准化
定义管理级【L4】	量化式管理【过程性能可预测】
优化级【L5】	持续优化

注：CMMI 另有连续式，其内容本质上与阶段式一致。

CMMI 文件体系：顶层方针文件--过程文件--规程文件--模板类文件

5、统一过程（在软考中 UP、RUP 都指统一过程）典型特点是用例驱动、以架构为中心、迭代和增量。9 个核心工作流：业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理、环境。

6、敏捷开发是一种以人为核心、迭代、循序渐进的开发方法，适用于小团队和小项目，具有小步快跑的思想。常见的敏捷开发方法有极限编程法、水晶法、并列争球法和自适应软件开发方法。

7、典型的原型开发方法模型。适用于需求不明确的场景，可以帮助用户明确需求。可以分为【抛弃型原型】与【演化型原型】

8、构件的组装：

顺序组装：按顺序调用已经存在的构件，可以用两个已经存在的构件来创造一个新的构件。

层次组装：被调用构件的“提供”接口必须和调用构件的“请求”接口兼容。

叠加组装：多个构件合并形成新构件，新构件整合原构件的功能，对外提供新的接口。

组装可能出现 3 种不兼容：参数不兼容、操作不兼容、操作不完备。

9、【逆向工程恢复信息的方法】：

用户指导下的搜索与变换方法：实现级、结构级

变换式方法：实现级、结构级、功能级

基于领域知识的方法：功能级、领域级

铅板恢复法：实现级、结构级

与逆向工程相关的概念有重构、设计恢复、再工程和正向工程。

（1）重构/重组（restructuring）。重构是指在【同一抽象级别】上转换系统描述形式。

（2）设计恢复（design recovery）。设计恢复是指借助工具从已有程序中抽象出有关数据设计、总体结构设计和过程设计等方面的信息。

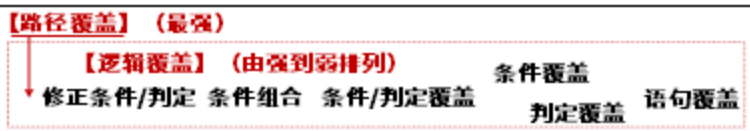
（3）逆向工程（reverse engineering）：逆向工程是分析程序，力图在比源代码更高抽象层次上建立程序的表示过程，逆向工程是设计的恢复过程。

（4）正向工程（forwardengineering）。正向工程是指不仅从现有系统中恢复设计信息，而且使用该信息去改变或重构现有系统，以改善其整体质量。

（5）再工程/重构工程（Re-engineering）。再工程是对现有系统的重新开发过程，包括逆向工程、新需求的考虑过程和正向工程三个步骤。

10、黑盒测试、白盒测试与灰盒测试：

白盒测试【结构测试】：关注内部结构与逻辑。

控制流分析、数据流分析、路径分析、程序变异【错误驱动测试】


黑盒测试【功能测试】：关注输入输出及功能。

等价类划分：不同等价类，揭示不同问题；有效等价类/无效等价类。
边界值分析：1≤x≤10，可取 x 的值为 0、1、10 和 11 作为测试数据。
错误推测：依靠测试人员的经验和直觉。
判定表：最适合描述在多个逻辑条件取值的组合所构成的复杂情况下，分别要执行哪些不同的动作。
因果图：根据输入条件与输出结果之间的因果关系来设计测试用例。

灰盒测试。灰盒测试介于黑盒与白盒测试之间。灰盒测试除了重视输出相对于输入的正确性，也看重其内部的程序逻辑。但是，它不可能像白盒测试那样详细和完整。它只是简单地靠一些象征性的现象或标志来判断其内部的运行情况，因此在内部结果出现错误，但输出结果正确的情况下可以采取灰盒测试方法。

11、软件测试阶段

单元测试：依据【详细设计】，模块测试，模块功能、性能、接口等。

集成测试：依据【概要设计】，模块间的接口。

系统测试：依据【需求文档】，包括功能测试、性能测试、验收测试、压力测试等。

确认测试：依据【需求文档】，验证软件与需求的一致性。内部确认测试、Alpha 测试、Beta 测试、验收测试。

12、其他测试：

其他测试	描述
AB 测试	多版本同时使用，有利于收集各版本的用户反馈，评估出最好版本。故也算是一种【网页优化方法】。
Web 测试	Web 系统测试与其他系统测试测试内容基本相同，只是测试重点不同。 Web 代码测试包括：源代码规则分析、链接测试、框架测试、表格测试、图形测试等方面。
链接测试	链接测试可分为 3 个方面： 1、测试所有链接是否按指示的那样确实链接到了该链接的页面。 2、测试所链接的页面是否存在。

	3、保证 Web 应用系统上没有孤立的页面。
表单测试	验证服务器是否能正确保存这些数据，后台运行的程序能否正确解释和使用这些信息。测试提交操作的完整性。
回归测试	测试软件变更之后，变更部分的正确性和对变更需求的符合性。

13、软件维护类型：

正确性维护【修 BUG】：识别和纠正软件错误/缺陷，测试不可能发现所有错误。

适应性维护【应变】：指使应用软件适应环境变化【外部环境、数据环境】而进行的修改。

完善性维护【新需求】：为扩充功能和改善性能而进行的修改。

预防性维护【针对未来】：为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使系统适应各类变化而不被淘汰。经典实例：【专用】改【通用】。

14、架构的本质：

（1）软件架构为软件系统提供了一个结构、行为和属性的高级抽象。

（2）软件架构风格是特定应用领域的惯用模式，架构定义一个词汇表和一组约束。

15、五大架构风格：

数据流风格：适合于分阶段做数据处理，交互性差，包括：批处理序列、管道过滤器。

调用/返回风格：一般系统都要用到，包括：主程序/子程序，面向对象，层次结构（分层越多，性能越差）。

独立构件风格：构件是独立的过程，连接件是消息传递。包括：进程通信，事件驱动系统（隐式调用）。

虚拟机风格：包括解释器与基于规则的系统，有自定义场景时使用该风格。

仓库风格（以数据为中心的风格）：以共享数据源为中心，其它构件围绕中心进行处理。包括：数据库系统、黑板系统（语言处理，信号处理），超文本系统。

16、MVC：

Model（模型）是应用程序中用于处理应用程序数据逻辑的部分。通常模型对象负责在数据库中存取数据。

View（视图）是应用程序中处理数据显示的部分。通常视图是依据模型数据创建的。

Controller（控制器）是应用程序中处理用户交互的部分。通常控制器负责从视图读取数据，控制用户输入，并向模型发送数据。

J2EE 体系结构中：视图（View）：JSP；控制（Controller）：Servlet；模型（Model）：Entity Bean、Session Bean。

17、SOA：

粗粒度，松耦合，标准化。Webservice 与 ESB 是 SOA 的实现技术。其中：

【UDDI】是 Web 服务集成的一个体系框架，包含了服务描述与发现的标准规范。

【WSDL】服务描述语言，三个基本属性（服务做些什么/如何访问服务/服务位于何处。

【SOAP】基于 XML 的协议，在分布式环境中交换信息。4 个部分：SOAP 封装（消息内容，谁发的，谁接收）、SOAP 编码规则（数据类型实例）、SOAP RPC（远程过程调用和应答协定）、SOAP 绑定（使用底层协议交换信息）。

18、【六边形架构（Hexagonal Architecture）】，又被称为端口与适配器架构（Ports and Adapters Architecture），由阿里斯泰尔·考本（Alistair Cockburn）在 2005 年提出。它是一种将系统核心业务逻辑与外部基础设施（如数据库、用户界面、第三方服务）彻底解耦的软件架构设计模式。

注：并不是因为系统刚好有六个部分，而是发明者用六边形来隐喻：核心业务可以通过多个不同的“面”（端口）与外界各种各样的技术进行平等交互。

19、REST（Representational State Transfer，表述性状态转移）：

REST 是一种通常使用 HTTP 和 XML 进行基于 Web 通信的技术，可以降低开发的复杂性，提高系统的可伸缩性。

REST 的 5 大原则：所有事物抽象为资源、资源唯一标识、通过接口操作资源、操作不改变资源标识、操作无状态。

REST API：

命令	用途
GET	从指定的资源请求数据，只进行数据检索，不进行其他操作
POST	将数据发送到服务器进行创建，通常用于上传文件或提交表单
PUT	更新目标资源的所有当前表示，使用上传的内容进行替换
DELETE	删除指定的资源
HEAD	与 GET 方法类似，但只传输状态行和头部信息
PATCH	对资源进行部分修改

20、微服务：

特点：小，且专注于做一件事情；轻量级的通信机制；松耦合、独立部署。

优势：技术异构性、弹性、扩展、简化部署、与结构相匹配、可组合性、对可替代性的优化。

微服务与 SOA 对比：

微服务	SOA
能拆分的就拆分	是整体的，服务能放一起的都放一起
纵向业务划分	是水平分多层
由单一组织负责	由按层级划分不同部门的组织负责
细粒度	粗粒度
两句话可以解释明白	几百字只相当于 SOA 的目录
独立的子公司	类似大公司里面划分了一些业务单元（BU）
组件小	存在较复杂的组件
业务逻辑存在于每一个服务中	业务逻辑横跨多个业务领域

使用轻量级的通信方式，如 HTTP	企业服务总线（ESB）充当了服务之间通信的角色

21、MDA 的核心模型：计算无关模型（CIM），平台独立模型（PIM），平台相关模型（PSM），代码 Code。

22、ADL 的三个基本元素：

构件：计算或数据存储单元；

连接件：用于构件之间交互建模的体系结构构造块及其支配这些交互的规则；

架构配置：描述体系结构的构件与连接件的连接图。

23、DSSA：

基本活动：领域分析（建立领域模型），领域设计（获得 DSSA），领域实现（开发和组织可复用信息）。

DSSA 角色：领域专家（有经验的用户、分析、设计、实现人员，“给建议”），领域分析人员（有经验的分析师，完成领域模型），领域设计人员（有经验的设计师，完成 DSSA），领域实现人员（有经验的程序员完成代码编写）。

24、ABSD：

ABSD 方法是架构驱动，即强调由业务、质量和功能需求的组合驱动架构设计。

ABSD 方法有三个基础：功能的分解，通过选择架构风格来实现质量和业务需求，软件模板的使用。

ABSD 开发过程：

（1）架构需求（需求获取、生成类图、对类进行分组、打包成构件、需求评审）

（2）架构设计（提出架构模型、映射构件、分析构件相互作用，产生架构，设计评审）

（3）架构文档化：从使用者角度编写，分发给所有相关开发人员，保证开发者手中版本最新。

（4）架构复审：标识潜在的风险，及早发现架构设计中的缺陷和错误。

（5）架构实现（复审后的文档化架构，分析与设计，构件实现，构件组装，系统测试）

（6）架构演化（需求变化归类，架构演化计划，构件变动，更新构件相互作用，构件组装与测试，技术评审，演化后的架构）

25、基于软件系统的生命周期，可以将软件系统的质量属性分为开发期质量属性和运行期质量属性 2 个部分。

开发期质量属性主要指在软件开发阶段所关注的质量属性，主要包含 6 个方面：

易理解性、可扩展性、可重用性、可测试性、可维护性、可移植性。

运行期质量属性主要指在软件运行阶段所关注的质量属性，主要包含 7 个方面：

性能、安全性、可伸缩性、互操作性、可靠性、可用性、鲁棒性。

26、场景：

场景是从风险承担者的角度与系统交互的简短描述。场景可从 6 个方面进行描述：刺激源、刺激、制品、环境、响应、响应度量。

27、架构评审四大质量属性：

（1）性能：代表参数（响应时间、吞吐量），设计策略（优先级队列、资源调度）。

（2）可用性：尽可能少的出错与尽快的恢复。代表参数（故障间隔时间，故障修复时间），设计策略（冗余、心跳线）。

（3）安全性：破坏机密性、完整性、不可否认性及可控性等特性。设计策略（追踪审计）

（4）可修改性：新增功能多少人·月能完成，设计策略（信息隐藏）

28、风险点：系统架构风险是指架构设计中潜在的、存在问题的架构决策所带来的隐患。

非风险点：一般以某种做法，“是可以实现的”、“是可以接受的”方式进行描述。

敏感点：指为了实现某种特定的质量属性，一个或多个构件所具有的特性。

权衡点：影响多个质量属性的特性，是多个质量属性的敏感点。

29、基本场景的评估方法：ATAM，SAAM，CBAM。

其中（1）SAAM：最初用于分析架构可修改性，后扩展到其他质量属性。

SAAM 五个步骤：即场景开发、体系结构描述、单个场景评估、场景交互和总体评估。

（2）ATAM：在 SAAM 的基础上发展起来的，主要针对性能、实用性、安全性和可修改性，在系统开发之前，对这些质量属性进行评价和折中。

ATAM 四大阶段：场景和需求收集、结构视图场景实现、属性模型构造和分析、折中。

（3）成本效益分析法（CBAM）是在 ATAM 基础上建立的，软件的“经济”模型。根据其投资回报选择架构策略。

（4）其它评估方法

【SAEM 方法】将软件架构看作一个最终产品以及设计过程中的一个中间产品，从外部质量属性和内部质量属性两个角度来阐述它的评估模型。

【SAABNet 方法】软件架构定性的评估技术依赖于专家知识，包括某些特定类型问题的解决方案以及可能的诱导因素、统计知识、审美观等，这些定性的知识比较含糊且难以文档化。该方法来源于人工智能(AI)，允许不确定、不完整知识的推理。

【SACMM 方法】是一种软件架构修改的度量方法，首先基于图内核定义差异度量准则来计算两个软件架构之间的距离。

【SASAM】SASAM 方法通过对预期架构和实际架构进行映射和比较来静态地评估软件架构，并将静态评估与架构开发方法 PuLSE-DSSA 结合，识别出 10 种不同的目的和需求来指导静态的架构评估。

【ALRRA 方法】可靠性风险主要包含两个因素：故障发生的可能性以及故障所致后果的严重性。ALRRA 是一种软件架构可靠性风险评估方法，该方法使用动态复杂度准则和动态耦合度准则来定义组件和连接件的复杂性因素。

【AHP 方法】是由美国运筹学家 T.L.Saaty 提出，它是对定性问题进行定量分析的一种简便、灵活而又实用的多准则决策方法。

【COSMIC+UML 方法】基于面向对象系统源代码的可维护性度量准则，Anjos 等人提出了基于度量模型来评估软件架构可维护性的方法。针对不同表达方式的软件架构，采用统一的软件度量 COSMIC 方法来进行度量和评估。

30、质量效用树：

ATAM 方法采用效用树（Utility tree）这一工具来对质量属性进行分类和优先级排序。效用树的结构包括：树根—质量属性—属性分类—质量属性场景（叶子节点）。

得到初始的效用树后，需要修剪这棵树，保留重要场景（通常不超过 50 个），再对场景按重要性给定优先级（用 H/M/L 的形式），再按场景实现的难易度来确定优先级（用 H/M/L 的形式），这样对所选定的每个场景就有一个优先级对（重要度、难易度），如（H、L）表示该场景重要且易实现。

31、负载均衡技术：

（1）应用层负载均衡：http 重定向、反向代理服务器；（2）传输层负载均衡：DNS 域名解析负载均衡、基于 NAT 的负载均衡；（3）硬件负载均衡：F5；（4）软件负载均衡：LVS、Nginx、HAproxy。

32、负载均衡算法：

现有的负载均衡算法主要分为静态和动态两类。静态负载均衡算法以固定的概率分配任务，不考虑服务器的状态信息；动态负载均衡算法以服务器的实时负载状态信息来决定任务的分配。

静态算法（不考虑动态负载）：

（1）轮转算法：轮流将服务请求（任务）调度给不同的节点（即：服务器）。

（2）加权轮转算法：考虑不同节点处理能力的差异。

（3）源地址哈希散列算法：根据请求的源 IP 地址，作为散列键从静态分配的散列表找出对应的节点。

（4）目标地址哈希散列算法：根据请求目标 IP 做散列找出对应节点。

（5）随机算法：随机分配，简单，但不可控。

动态算法（考虑动态负载）

（1）最小连接数算法：每个节点处理能力相同的情况下，新请求分配给当前活动请求数量最少的节点。

（2）加权最小连接数算法：考虑节点处理能力不同，按最小连接数分配。

（3）加权百分比算法：考虑了节点的利用率、硬盘速率、进程个数等，使用利用率来表现剩余处理能力。

还有基于场景的负载均衡策略：基于场景的负载均衡结合业务特性或外部条件（如时间段、地理位置、请求类型）动态选择策略，实现精细化控制。常见策略包括：

（1）动态流量分配：实时监控服务器负载指标（如 CPU 利用率、请求响应时间），结合预设阈值自动调整流量分发权重。

（2）资源感知调度：根据服务器资源类型（如 GPU 加速、高内存）和任务计算密度自动选择节点。

（3）地理位置优化：根据用户地理位置将请求导向最近的边缘节点。

（4）业务优先级调度：将核心服务（如支付、登录）分配到独立高可用集群，与普通查询服务物理隔离。

33、边云协同的优缺点：

边云协同的优点：

（1）超低延迟与实时响应

大量数据不需要传到遥远的云端，在本地边缘节点就能完成处理和反馈。

效果：对于自动驾驶、工业机器人控制、智能电网等对时间极度敏感的场景，能实现毫秒级的响应。

（2）节省网络带宽与成本

边缘设备对海量原始数据进行预处理和过滤，只将必要的数据上传到云端。

效果：极大地减少了海量数据涌入骨干网络带来的带宽压力，降低了网络流量成本。

（3）提高可靠性与业务连续性（离线自治）

边缘节点具备一定的独立计算和存储能力。

效果：即使外部网络中断（与云端失联），本地边缘系统依然能依靠“离线自治”维持核心功能的正常运转，不会导致整个系统瘫痪。

（4）增强数据隐私与安全性

敏感数据可以直接在本地边缘端处理并销毁，不需要上传到公有云。

效果：从源头上减少了数据在网络传输过程中被窃听或泄露的风险。

边云协同的缺点与挑战：

（1）系统复杂度较高

需要同时管理海量的、异构的边缘设备和统一的云端服务器。

效果：架构设计、资源调度、应用分发以及云边之间的数据同步都会变得复杂，研发和运维成本显著增加。

（2）边缘节点安全防护脆弱

云计算中心通常有严密的物理和网络防护，而边缘设备分布在外界，易遭受物理破坏或局部网络攻击。

效果：一旦某个边缘节点被攻破，可能会成为攻击整个系统的跳板。

（3）运维与管理困难

边缘设备地理位置分散，硬件配置可能千差万别（不同的芯片架构、不同的操作系统）。

效果：软件升级、固件更新、设备故障排查变得麻烦，难像纯云端那样一键批量自动化运维。

（4）资源受限导致能力有限

边缘节点（如嵌入式设备、小型服务器）的算力、内存和存储容量都是非常有限的。

效果：边缘无法运行大型的深度学习模型，必须依赖云端来做算法训练和策略下发。

其它：数据的“单向割裂”与一致性问题、初始建设与硬件采购成本较高。

34、Redis：

Redis 与 MemCache 能力比较

工作	MemCached	Redis
数据类型	简单 key/value 结构	丰富的数据结构
持久性	不支持	支持
分布式存储	客户端哈希分片/一致性哈希	多种方式，主从、Sentinel、Cluster 等
多线程支持	支持	支持（Redis5.0 及以前版本不支持）

内存管理	私有内存池/内存池	无
事务支持	不支持	有限支持
数据容灾	不支持，不能做数据恢复	支持，可以在灾难发生时，恢复数据

Redis 分布式存储方案

分布式存储方案	核心特点
主从（Master/Slave）模式	一主多从，故障时手动切换。
哨兵（Sentinel）模式	有哨兵的一主多从，主节点故障自动选择新的主节点。
集群（Cluster）模式	分节点对等集群，分 slots，不同 slots 的信息存储到不同节点。

35、主从数据库结构特点：

一般：一主多从，也可以多主多从。

主库做写操作，从库做读操作。

主从复制步骤：

主库（Master）更新数据完成前，将操作写入 binlog 日志文件。

从库（Slave）打开 I/O 线程与主库连接，做 binlog dump process，并将事件写入中继日志。

从库执行中继日志事件，保持与主库一致。

36、分表和分区：

	分区	分表
共性	1、都针对数据表　　2、都使用了分布式存储 3、都提升了查询效率　　4、都降低了数据库频繁 I/O 的压力值	
差异	逻辑上还是一张表	逻辑上已是多张表。

其中分区的优点：

（1）相对于单个文件系统或是硬盘，分区可以存储更多的数据。

（2）数据管理比较方便，比如要清理或废弃某年的数据，就可以直接删除该日期的分区数据即可。

（3）精准定位分区查询数据，不需要全表扫描查询，大大提高数据检索效率。

（4）可跨多个分区磁盘查询，来提高查询的吞吐量。

（5）在涉及聚合函数查询时，可以很容易进行数据的合并。

37、软件复用是一种系统化的软件开发过程，通过识别、分析、分类、获取和修改软件实体，以便在不同软件开发过程中重复使用它们。

软件开发过程中重复使用相同或相似软件元素的过程。

软件架构复用类型	机会复用	开发过程中，只要发现有可复用资产，就对其进行复用
	系统复用	开发之前，要进行规划，以决定哪些需要复用

软件架构复用的基本过程：复用的基本过程主要包括 3 个阶段：首先构造/获取可复用的软件资产，其次管理这些资产，最后针对特定的需求，从这些资产中选择可复用的部分，以开发满足需求的应用系统。

38、WPDRRC 模型：

WPDRRC 信息安全模型是我国“八六三”信息安全专家组提出的适合中国国情的信息系统安全保障体系建设模型。

WPDRRC 模型包括 6 个环节和 3 大要素。

6 个环节包括：预警、保护、检测、响应、恢复和反击。模型蕴涵的网络安全能力主要是预警能力、保护能力、检测能力、响应能力、恢复能力和反击能力。

3 大要素包括人员、策略和技术。

39、区块链的四大基础技术支柱以及关键支撑技术：

（1）区块链的四大基础技术支柱

分布式存储：简单来说，就是一种将数据分散存储到多个地方的数据存储技术，而且存储的数据可在多个参与者之间共享，人人可以参与，并具有相同的权力，一起记录数据，主要起到了数据存储的功能。

共识机制：因为在区块链的分布式网络中，没有中央权威。因此，网络需要一个决策机制来促成参与者达成一致。而共识机制就是一种协调大家处理数据的机制。共识机制就决定了这些数据中，谁获得数据的记账权。

智能合约：是一种旨在以信息化方式传播、验证或执行合同的计算机协议。智能合约可以在没有第三方的情况下，也能进行可信的交易，而且这些交易可追踪且不可逆转。所以，智能合约在系统中，主要起到了数据的执行作用。【执行智能合约的机制：去中心化网络+共识机制+受激励的矿工】

密码学：是一种特殊的加密和解密技术，区块链系统中，应用了多种多样的密码学技术，包括哈希算法、公钥私钥、数字签名等等，以此来保证整个系统的数据安全，并且证明了数据的归属。

（2）区块链的关键支撑技术

区块结构（Block Structure）：区块链中存储批量交易数据及验证信息的基本数据单元，由区块头和区块体组成，通过密码学方法链接形成不可篡改的链式结构。

P2P 网络（Peer-to-Peer Network）：一种去中心化的网络架构，其中所有节点（Peer）以平等地位互联，每个节点既能作为客户端请求服务，也能作为服务器提供资源或服务，不依赖中央服务器进行协调。

默克尔树（Merkle Tree）：一种基于密码学哈希函数构建的树形数据结构，通过逐层聚合子节点的哈希值，最终生成一个唯一的根哈希（Merkle Root），用于高效验证大规模数据的完整性。

40、国产密码算法：

算法名称	算法特性描述	备注
SM1	对称加密，分组长度和密钥长度都为 128 比特	广泛应用于电子政务、电子商务及国民经济的各个应用领域
SM2	非对称加密，用于公钥加密算法、密钥交换协议、数字签名算法	国家标准推荐使用素数域 256 位椭圆曲线
SM3	杂凑算法，杂凑值长度为 256 比特	适用于商用密码应用中的数字签名和验证
SM4	对称加密，分组长度和密钥长度都为 128 比特	适用于无线局域网产品

SM9	标识密码算法	不需要申请数字证书，适用于互联网应用的各种新兴应用的安全保障

41、嵌入式微处理器工作温度范围：

嵌入式微处理器主要用于处理相关任务。由于嵌入式系统通常都在室外使用，可能处于不同环境，因此选择处理器芯片时，也要根据不同使用环境选择不同级别的芯片。其主要因素是芯片可适应的工作环境温度。通常，我们把芯片分为民用级、工业级和军用级。

民用级器件的工作温度范围：0~70°C

工业级器件的工作温度范围：-40~85°C

军用级器件的工作温度范围：-55~150°C

当然，除了环境温度外，环境湿度、震动、加速度等也是应考虑的因素。

42、嵌入式软件的主要特点：

（1）可裁剪性：采用的设计方法包括静态编译、动态库和控制函数流程实现功能控制等

（2）可配置性：采用的设计方法包括数据驱动、静态编译和配置表等

（3）强实时性：采用的设计方法包括表驱动、配置、静/动态结合、汇编语言等

（4）安全性：采用的设计方法包括编码标准、安全保障机制、FMECA（故障模式、影响及危害性分析）等

（5）可靠性：采用的设计方法包括容错技术、余度技术和鲁棒性设计等

（6）高确定性：采用的设计方法包括静态分配资源、越界检查、状态机、静态任务调度等

43、关系型数据库和 NoSQL 对比：

对比维度	关系数据库	NoSQL
应用领域	面向通用领域	特定应用领域
数据容量	有限数据	海量数据
数据类型	结构化数据【二维表】	非结构化数据
并发支持	支持并发、但性能低	高并发
事务支持	高事务性	弱事务性
扩展方式	向上扩展	向外扩展

44、NoSQL 数据库分类：

分类	典型产品	数据模型	典型应用场景	分类	典型产品
键值对存储	Memcached、Redis	key-value 键值对	缓存、日志等	键值对存储	Memcached、Redis
列式存储	BigTable、Hbase、Cassandra	列簇式存储	分布式数据存储和管理	列式存储	BigTable、Hbase、Cassandra

文档型	MongoDB、CouchDB	JSON/BSON 文档	电商商品目录与属性管理、内容管理系统	文档型	MongoDB、CouchDB
图数据库	Neo4j、OrientDB	节点（Node）+边（Edge）	知识图谱存储、社交网络好友推荐、金融反欺诈（资金环路检测）、推荐系统	图数据库	Neo4j、OrientDB
时序数据库	InfluxDB、TDengine	时间戳+ 指标值	物联网、监控告警、金融量化等“时间戳驱动”场景	时序数据库	InfluxDB、TDengine
向量数据库	Milvus	高维向量	大模型 RAG、语义搜索、推荐系统等“AI 基础设施”场景	向量数据库	Milvus

NoSQL 数据库在以下这几种情况比较适用：（1）数据模型比较简单；（2）需要灵活性更强的 IT 系统；（3）对数据库性能要求较高；（4）不需要高度的数据一致性；（5）对于给定 key,比较容易映射复杂值的环境。

NoSQL 整体框架：

接口层(Interface)
数据逻辑层(Data Logical Model)
数据分布层(Data Distribution Model)
数据持久层(Data Persistence)

NoSQL 数据库与关系型数据库（如 MySQL）混用，数据一致性解决方案：

（1）实时同步方式，在数据查询时，首先从缓存中查找，如果查询不到再从 MySQL 数据库中查询，并将查询结果保存到缓存；更新数据时，首先更新数据库，再将缓存中相应数据设置为过期或失效，或者更新缓存中的相应数据。

（2）异步队列方式，比如采用消息中间件。

（3）采用自定义函数的方式，在主数据库中进行编程，利用触发器的方式进行数据同步。该方式实现简单，但对主数据库的性能影响大。

（4）使用专门数据同步工具，比如目前存在的 MySQL 日志同步工具，如 canal 等。

45、 Kappa 架构与 Lambda 的对比：

对比内容	Lambda 架构	Kappa 架构

复杂度与开发、维护成本	需要维护两套系统（引擎），复杂度高，开发、维护成本高	只需要维护一套系统（引擎），复杂度低，开发、维护成本低
计算开销	需要一直运行批处理和实时计算，计算开销大	必要时进行全量计算，计算开销相对较小
实时性	满足实时性	满足实时性
历史数据处理能力	批式全量处理，吞吐量大，历史数据处理能力强	流式全量处理，吞吐量相对较低，历史数据处理能力相对较弱
使用场景	直接支持批处理，更适合对历史数据分析查询的场景，期望尽快得到分析结果，批处理可以更直接高效地满足这些需求	不是 Lambda 的替代架构，而是简化，Kappa 放弃了对批处理的支持，更擅长业务本身为增量数据写入场景的分析需求
选择依据	根据两种架构对比分析，将业务需求、技术要求、系统复杂度、开发维护成本和历史数据处理能力作为选择考虑因素。 计算开销虽然存在一定差别，但是相差不是很大，所以不作为考虑因素	

46、CPS 的建设路径：CPS 体系设计→单元级 CPS 建设→系统级 CPS 建设→SoS 级 CPS 建设。

CPS 典型落地架构——感知层、边缘层、平台层、应用层

CPS 为实现“状态感知→实时分析→科学决策→精准控制”的数据闭环，通常采用分层解耦的云边协同架构：

层级	位置	核心职责	与数据闭环的对应
感知控制层	物理空间	通过传感器、执行器采集物理量，并执行控制指令	状态感知+精准控制
边缘层	靠近物理端	数据过滤、聚合、本地实时分析，实现断网自治	实时分析（近端）
平台层（云/AI 层）	信息空间	全局数据融合、AI 模型训练与推理、跨域协同决策	科学决策（远端）
应用层	用户交互端	提供可视化界面、预警通知、第三方系统集成	人机协同

关键特征：边缘层承担断网自治能力——当云边网络中断时，边缘网关仍可独立完成本地闭环控制。

47、人工智能是利用数字计算机或者数字计算机控制的机器模拟、延伸和扩展人的智能，感知环境、获取知识并使用知识获得最佳结果的理论、方法、技术及应用系统。

目标：了解智能的实质，并生产出一种新的能以人类智能相似的方式做出反应的智能机器。【图灵测试】

分类：弱人工智能和强人工智能【是否能真正实现推理、思考和解决问题】

关键技术：自然语言处理、计算机视觉、知识图谱、人机交互、虚拟现实/增强现实、机器学习。

48、世界模型：

【世界模型】是人工智能领域用于模拟环境动态并预测未来状态的核心技术框架，通过多模态数据训练构建系统内部表征。AI 在大脑中对真实世界运行规律构建的“虚拟模拟器”。

三大核心能力：

【生成性】指能生成语义、几何、物理一致的三维模拟环境。

【多模态】指可处理图像、视频、深度、文本等任意输入模态。

【互动性】指能根据动作预测系统状态变化，实现因果推理。

应用场景：

自动驾驶领域（特斯拉、小米训练自动驾驶算法）、机器人技术应用（模拟人机协作场景）、视频生成领域（符合物理定律）、科研医疗领域等。

49、机器学习

机器学习分类方式	类别	特点及算法
按学习模式分类	监督学习	需要提供标注的样本集。自然语言处理、信息检索、文本挖掘、手写体辨识、垃圾邮件侦测。 回归（线性回归、逻辑回归）、分类（决策树、SVM、KNN、朴素贝叶斯）。
	无监督学习	不需要提供标注的样本集。经济预测、异常检测、数据挖掘、模式识别。自监督学习是一种特殊的无监督学习方法。 聚类（Kmeans 算法【文本聚类】）、Apriori 算法、主成分分析。
	半监督学习	需要提供少量标注的样本和大量的未标注样本进行训练和分类。
	强化学习	学习从环境状态到行为的映射，需要反馈机制。无人驾驶、工业控制、机器人控制。
按学习方法分类	传统机器学习	传统机器学习的领域特征需要手动完成，且需要大量领域专业知识
	深度学习	不需要人工特征提取，但需要大量的训练数据集以及强大的 GPU 服务器来提供算力。 卷积神经网络（CNN）、循环神经网络（RNN）、Transformer。
机器学习的常见算法还包括迁移学习、主动学习和演化学习。		

50、多模态

【多模态】在计算机科学、人工智能和机器学习领域，多模态技术指的是通过整合来自不同模态的数据（如图像、文字、音频、视频等），从而增强模型的理解能力和推理能力。

【特征提取与表示学习】目的是将不同模态的异构数据转换为模型能够处理的向量或特征空间表示。

【模态对齐】目的是使不同模态的数据在时空和语义上能够相互关联。对齐技术主要通过在不同模态间找到相同的语义概念或时空位置，以保证融合的数据具有一致性。

【数据融合】融合来自不同模态的数据（如图像、文本、音频等），弥补单一模态在任务中的不足，获得更全面的感知能力。数据融合有不同层次，包括数据级、特征级和决策级。

【多模态融合与推理】结合来自多个模态的特征，为模型提供更高的理解能力，能够实现如视觉问答、情感分析等高级应用。

【跨模态转换（可选）】在不同模态之间进行内容生成或转化，例如将文本描述生成相应的图像，或通过图像生成文字描述。

51、人工智能芯片

人工智能芯片四大类（按技术架构**分类**）：GPU、FPGA（现场可编程门阵列）、ASIC（专用集成电路）、类脑芯片。

AI 芯片的关键特征：

新型的计算范式：AI 计算既不脱离传统计算，也具有新的计算特质

训练和推断：AI 系统通常涉及训练和推断过程

大数据处理能力：满足高效能机器学习的数据处理要求

数据精度：降低精度的设计

可重构的能力：针对特定领域而不针对特定应用的设计，可以通过重新配置，适应新的 AI 算法、架构和任务

52、数字孪生体的三项核心技术：建模、仿真和基于数据融合的数字线程。

能够做到统领建模、仿真和数字线程的系统工程和 MBSE，则称为数字孪生体的顶层框架技术。

物联网是数字孪生体的底层伴生技术。

云计算、机器学习、大数据、区块链则成为数字孪生体的外围使能技术。

53、数字孪生生态系统

行业应用层	包括智能制造、智慧城市在内的多方面应用
共性应用层	包括描述、诊断、预测、决策四个方面
模型构建与仿真分析层	包括数据建模、数据仿真和控制
数据互动层	包括数据采集、数据传输和数据处理等内容
基础支撑层	由具体的设备组成

54、使用许可：

按照被许可使用权的排他性强弱不同，可以将使用许可分为以下三种：

①独占使用许可—仅 1 个授权对象可用，著作权人不可用。

②排他使用许可—仅 1 个授权对象和著作权人可用。

③普通使用许可—多个授权对象和著作权人可用。

55、保护期限

客体类型	权利类型	保护期限
公民作品	**署名权、修改权、保护作品完整权**	**没有限制**
	发表权、使用权和获得报酬权	作者终生及其死亡后的 50 年（第 50 年的 12 月 31 日）
单位作品	发表权、使用权和获得报酬权	50 年（首次发表后的第 50 年的 12 月 31 日）
公民软件产	**署名权、修改权**	**没有限制**

品	发表权、复制权、发行权、出租权、信息网络传播权、翻译权、使用许可权、获得报酬权、转让权	作者终生及其死后 50 年（第 50 年的 12 月 31 日）。
单位软件产品	发表权、复制权、发行权、出租权、信息网络传播权、翻译权、使用许可权、获得报酬权、转让权	50 年（首次发表后的第 50 年的 12 月 31 日）
注册商标		有效期 10 年（若注册人死亡或倒闭 1 年后，未转移则可注销，期满后 6 个月内必须**续注**）
发明专利权		保护期为 20 年（从申请日开始）
实用新型专利权		保护期为 10 年（从申请日开始）
外观设计专利权		保护期为 15 年（从申请日开始）
商业秘密		**不确定**，公开后公众可用

软考刷题系统
刷题打卡,试题含视频解析

备考资料下载
备考大全/希赛备考资料